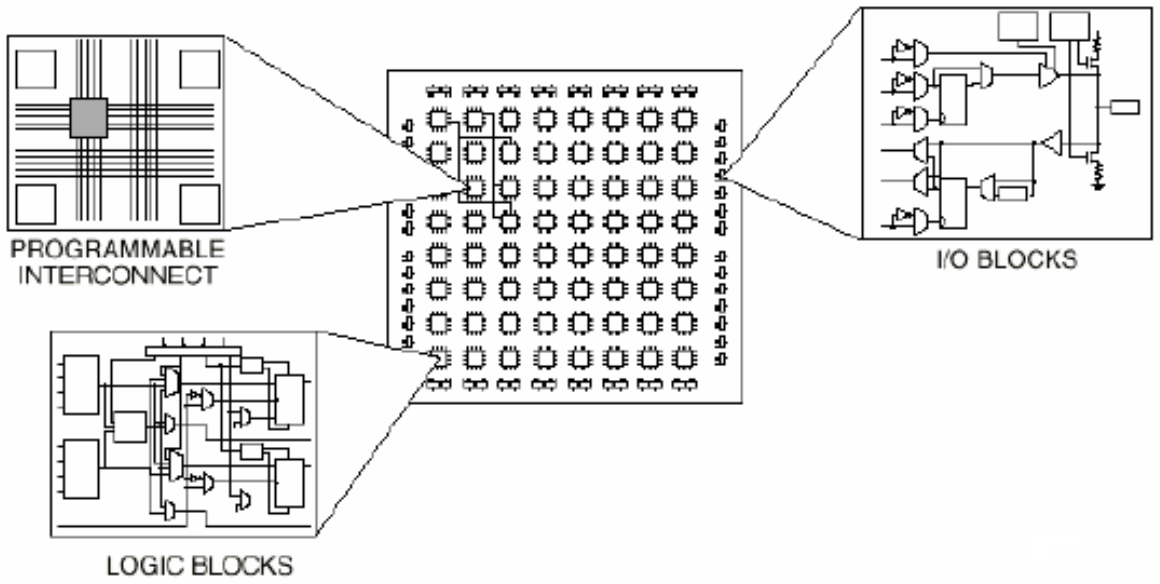
2017

Circuit Logique Programmable



M. Probus A. F. KIKI

Liste des figures

1.1 Logique à diodes 2
1.2 Logique à transistors 2
1.3 Structure électronique ET à fusible 3
1.4 Structure électronique ET à transistor 3
1.5 Structure électronique OU 4
1.6 symbolisation des portes logiques pour les PLDs 4
1.7 Structure de base d'un PLDs 5
1.8 Structure de base avec les normes des constructeurs 5
1.9 PLD programmé 6
1.10 Resumé graphique des PLD 9
1.11 PAL, Structure à OU fixe et ET Programmable 10
1.12 PROM, Structure à ET fixe et OU Programmable 10
1.13 Exercice PROM 11
1.14 Exercice PAL 11
1.15 configuration des entrée/sortie 12
1.16 Configuration partielle interne du PAL 16L8 14
1.17 Brochage du PAL 16L8 14
1.18 Brochage du PAL 16R6 15
1.19 Configuration partielle interne du PAL 16R8 16

Liste des tableaux

1.1 Liste différentes familles de PLD 8

Contenu

Contenu . . . . . **iii**

1 Présentation des circuits logiques programmables **1**

1.1 Rappel sur les principes électronique des portes ET et OU . 1

1.1.1 Logique à diodes 1

1.1.2 Logique à transistors 2

1.2 Structure de base d'un PLD 2

1.2.1 Structure «Et» 3

1.2.2 Structure «OU» 4

1.3 Les différentes familles de PLD 7

1.3.1 Illustration des structure à «ET» programmable et «OU» fixe 10

1.3.2 Illustration des structure à «ET» fixe et «OU» programmable 10

1.4 Les circuits PAL 11

1.4.1 Présentation 11

1.4.2 Référence des PAL (selon AMD) 12

1.4.3 Exemple de PAL 13

1.4.3.1 Le PAL 16L8 14

1.4.3.2 Le PAL 16R8 15

1.5 Programmation des PLD 16

2 Initiation au Language VHDL 18
2.1 Structure d'une description VHDL simple 19
2.1.1 Déclaration des bibliothèques 20
2.1.2 Déclaration de l'entité et des entrées/sorties (I/O) . . 21
2.1.2.1 NOM DU SIGNAL 21
2.1.2.2 Le sens du signal 22
2.1.2.3 Le type 22
2.1.2.4 Exemples de description d'entités 23
2.1.3 Déclaration de l'architecture correspondante à l'entité 24
2.2 Les instructions de base (mode « concurrent »), logique combinatoire 25
2.2.1 Les opérateurs 26
2.2.1.1 L'affectation simple : <= 26
2.2.1.2 Opérateur de concaténation 27
2.2.1.3 Opérateurs logiques 28
2.2.1.4 Opérateurs arithmétiques 28
2.2.1.5 Opérateurs relationnels 29
2.2.2 Les instructions du mode « concurrent » 29
2.2.2.1 Affectation conditionnelle 29
2.2.2.2 Affectation sélective 30
2.3 Les instructions du mode séquentiel 30
2.3.1 Définition d'un PROCESS 30
2.3.2 Les deux principales structures utilisées dans un process 31
2.3.3 Exemples de process 31
2.4 Les compteurs 32
2.5 Installation du logiciel de Programmation VHDL 33
2.6 Installation du logiciel de Simulation 33
2.7 Travaux pratiques de Programmation et de Simulation . . . 33

.

CHAPITRE 1

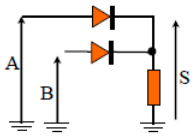
Présentation des circuits logiques programmables

Introduction

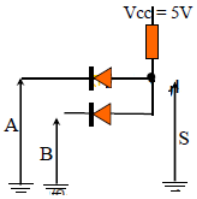
Il y a quelques années la réalisation d'un montage en électronique numérique impliquait l'utilisation d'un nombre important de circuits intégrés logiques. Ceci avait pour conséquences un prix de revient élevé, une mise en œuvre complexe et un circuit imprimé de taille importante. Le développement des mémoires utilisées en informatique fut à l'origine des premiers circuits logiques programmables (PLD : Programmable Logic Device). Ce type de produit peut intégrer dans un seul circuit plusieurs fonctions logiques programmables par l'utilisateur. Sa mise en œuvre se fait très facilement à l'aide d'un programmateur, d'un micro-ordinateur et d'un logiciel adapté.

1.1 Rappel sur les principes électronique des portes ET et OU

1.1.1 Logique à diodes



A (en V)	B (en V)	S (en V)
0	0	0
0	5	5
5	0	5
5	5	5

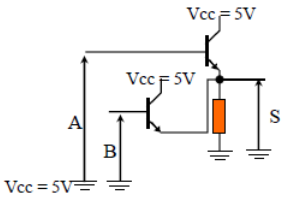


A (en V)	B (en V)	S (en V)
0	0	0
0	5	0
5	0	0
5	5	5

Figure 1.1 – Logique à diodes

1.1.2 Logique à transistors

A (en V)	B (en V)	S (en V)
0	0	0
0	5	5
5	0	5
5	5	5



A (en V)	B (en V)	S (en V)
0	0	0
0	5	0
5	0	0
5	5	5

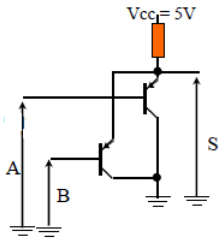


Figure 1.2 – Logique à transistors

1.2 Structure de base d'un PLD

La plupart des PLDs suivent la structure suivante :

- Un ensemble d'opérateurs « ET » sur lesquels viennent se connecter les variables d'entrée et leurs compléments ;
- Un ensemble d'opérateurs « OU » sur lesquels les sorties des opérateurs « ET » sont connectées ;
- Une éventuelle structure de sortie (Portes inverseuses, logique 3 états, registres...).

Les deux premiers ensembles forment chacun ce qu'on appelle une **matrice**. Les interconnexions de ces matrices doivent être programmables. C'est la raison pour laquelle elles sont assurées par des **fusibles** qui sont « grillés » lors de la programmation. Lorsqu'un PLD est vierge toutes les connexions sont assurées.

1.2.1 Structure «Et»

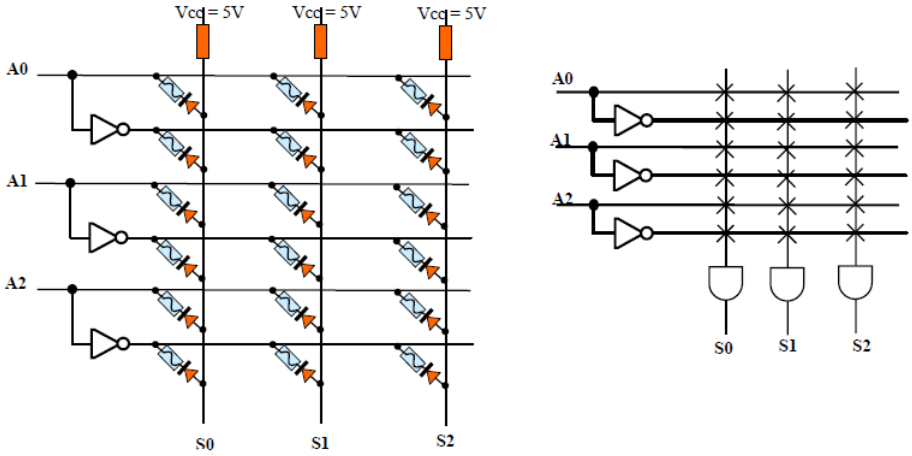


Figure 1.3 – Structure électronique ET à fusible

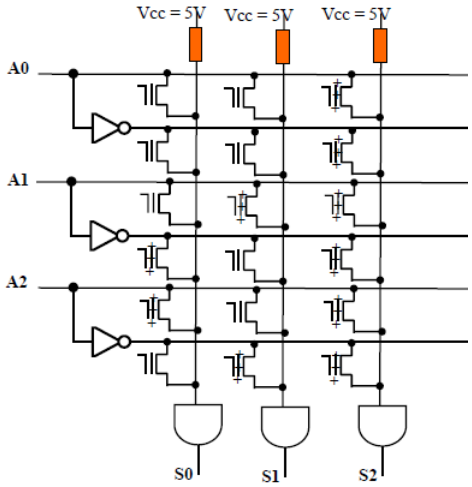


Figure 1.4 – Structure électronique ET à transistor

1.2.2 Structure «OU»

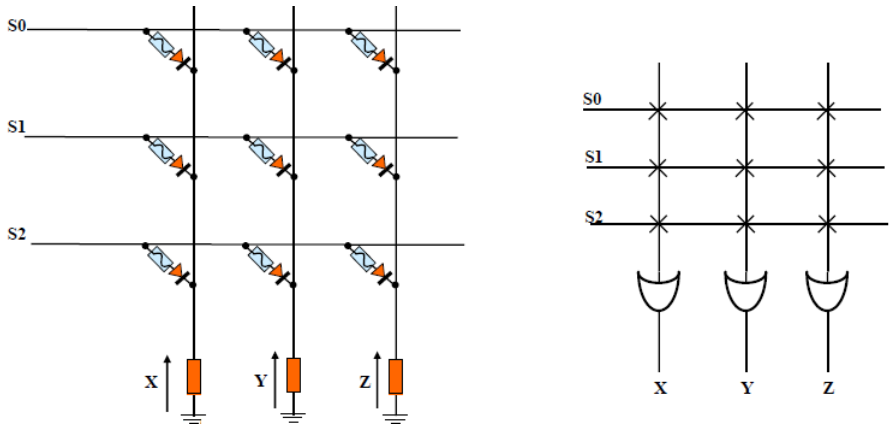


Figure 1.5 – Structure électronique OU

La figure 1.7 présente un exemple de structure d'un PLD. On peut remarquer que la représentation d'une telle structure est complexe alors que le nombre de portes intégrées est peu important. Les constructeurs ont donc très rapidement adoptés un autre type de représentation rendant les schémas beaucoup plus lisibles. On remarquera que la norme adoptée est américaine (normal, la plupart des constructeurs sont américains). Un exemple de cette représentation est donné par la figure 1.8.

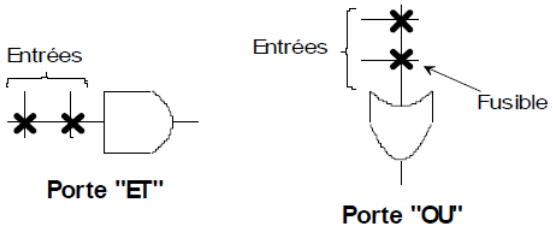


Figure 1.6 – symbolisation des portes logiques pour les PLDs

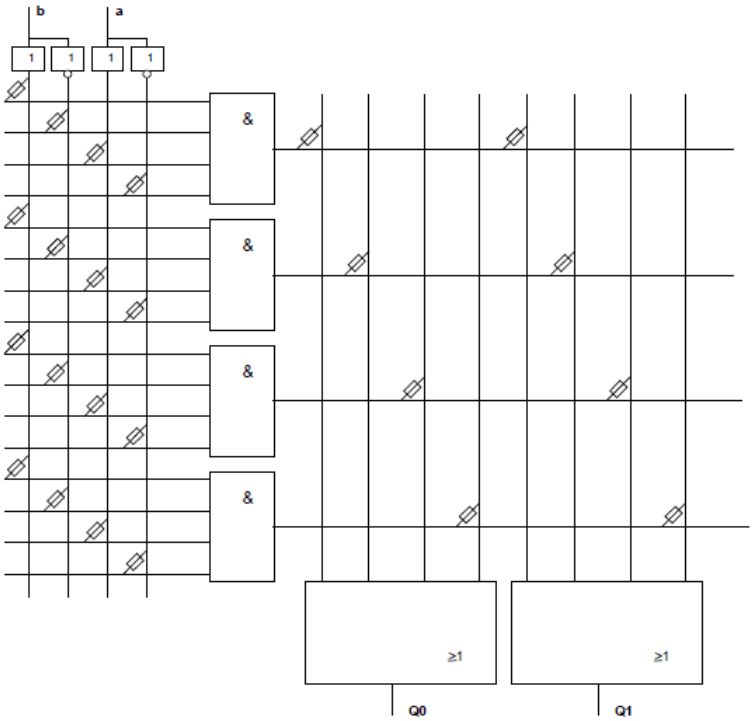


Figure 1.7 – Structure de base d'un PLDs

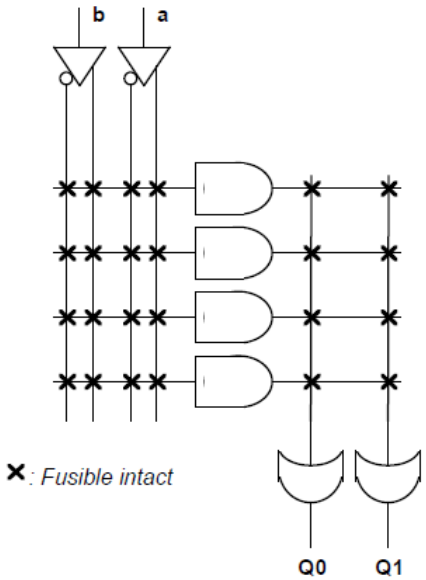


Figure 1.8 – Structure de base avec les normes des constructeurs

La figure 1.8 représente la structure interne d'un PLD ayant ses fusibles intacts. Les équations logiques de Q0 et Q1 sont :

$$Q0 = Q1 = a.\overline{a}.b.\overline{b} + a.\overline{a}.b.\overline{b} + a.\overline{a}.b.\overline{b} + a.\overline{a}.b.\overline{b} = 0$$

Question : On souhaite obtenir les fonctions suivantes.

$$Q0 = a.\overline{b} + \overline{a}.b \; et \; Q1 = \overline{a}.\overline{b} + a.b$$

Quels sont les fusibles qu'on « grillera » ?

Réponse : Les fusibles seront griller de façon à obtenir le schéma suivant :

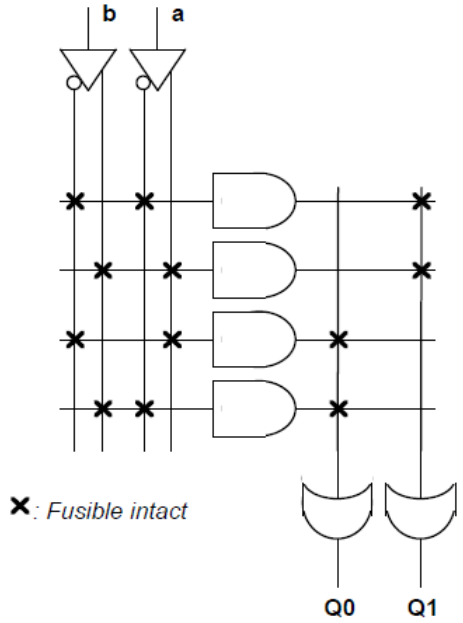


Figure 1.9 – PLD programmé

1.3 Les différentes familles de PLD

Les constructeurs de PLD se livrent entre eux à une guerre commerciale et sur les appellations, ce qui explique la difficulté à établir la classification des PLD La plus ancienne et la plus connue est certainement la famille des PAL Le nom a été donné par la société MMI (Fusion depuis avec AMD), c'est une appellation déposée, comme Walkman pour Sony ou Réfrigérateur pour Frigidaire.

* PAL signifie Programmable Array Logic, c'est à dire réseau logique programmable. La programmation de ces circuits s'effectue par destruction de fusibles. Une fois programmés on ne peut plus les effacer. On distingue deux sous familles :

- Les PAL combinatoires ou PAL simples. Ils sont constitués de fonctions de logique combinatoire.
- Les PAL à registres ou FPLS Field Programmable Logic Sequencer pour séquenceur logique programmable. Ils sont constitués de logique combinatoire et séquentielle (Registre).

* Les PAL effaçables : EPLD

Les EPLD Ce qui signifie Erasable Programmable Logic Device, c'est à dire circuit logique programmable et effaçable et qui sont aux PAL ce que sont les UVPROM aux PROM. Les EPLD peuvent être effacés par U.V. ou électriquement. Ils sont encore appelés PAL CMOS.

* Les GAL Les GAL signifie Generic Array Logic ou encore réseau logique générique ce qui veut dire pas grand chose mais qui sont aux PAL ce que sont les EEPROM aux PROM. Le nom de GAL a été déposé par LATTICE SEMICONDUCTOR. Leur fonctionnement est identique aux PAL CMOS, ils sont programmables et effaçables électriquement.

* Les CPLD

Les CPLD signifie Complex Programmable Logic Device. Ces circuits sont composés de plusieurs PAL élémentaires (Par exemple l'équivalent de PAL 22V10) reliés entre-eux par une zone d'interconnexion. Grâce à cette architecture, ils permettent d'atteindre des vitesses de fonctionnement élevées (plusieurs centaine de Mhz).

* Les LCA et FPGA à anti-fusible.

– Les LCA Ce qui signifie Logic Cell Array ou encore réseau de cellules logiques. Ces circuits sont composés de blocs logiques élémentaires de 2000 à 10000 portes que l'utilisateur peut interconnecter.

– Les FPGA à anti fusibles sont identiques aux LCA sauf qu'ils permettent une plus grande intégration de portes et ils ne sont pas effaçables électriquement. Le nom anti-fusible vient de la programmation des connexions qui s'effectue par fermeture de circuits, comparé aux fusibles où l'on ouvre les circuits.

On peut résumé dans le tableau 1.1 la structure des différents PLD, et sur la figure 1.10 les différentes catégories de PLD

Tableau 1.1 – Liste différentes familles de PLD

TYPE	Nombre de portes intégrées	Matrice ET	Matrice OU	Effaçable
PROM	2 000 à 500 000	Fixe	Programmable	Non
PAL	10 à 100	Programmable	Fixe	Non
GAL	10 à 100	Programmable	Fixe	Electriquement
EPLD	100 à 3000	Programmable	Fixe	Aux U-V
FPLA	2000 à 3000	Programmable	Programmable	Electriquement

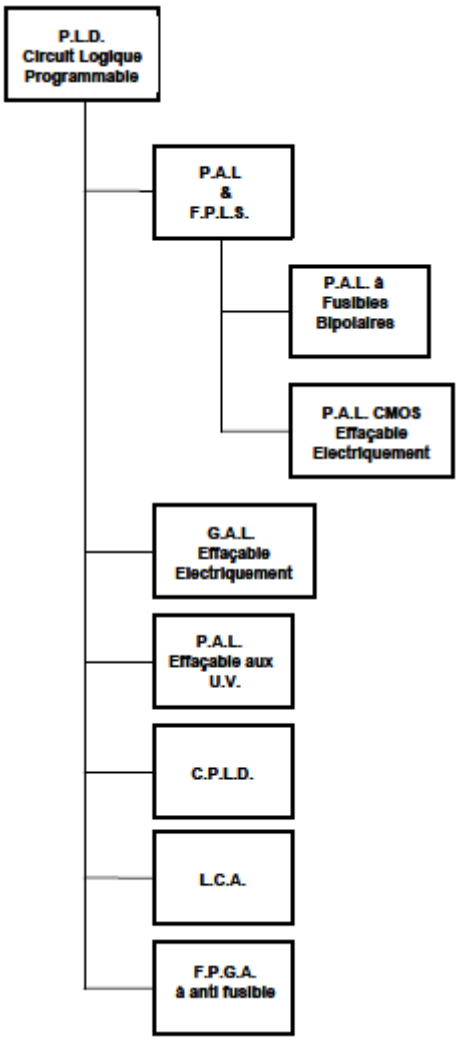


Figure 1.10 – Resumé graphique des PLD

1.3.1 Illustration des structure à «ET» programmable et «OU» fixe

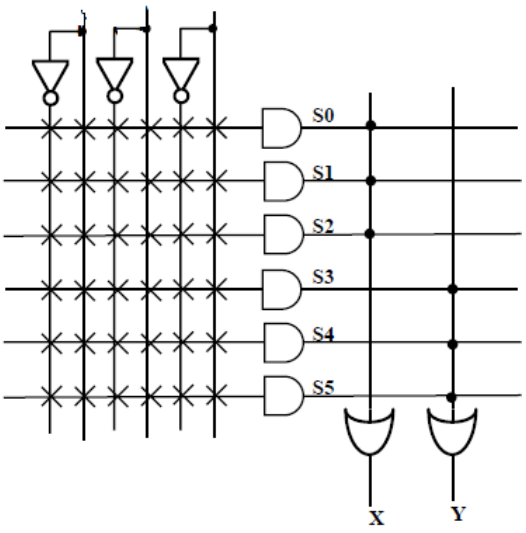


Figure 1.11 – PAL, Structure à OU fixe et ET Programmable

1.3.2 Illustration des structure à «ET» fixe et «OU» programmable

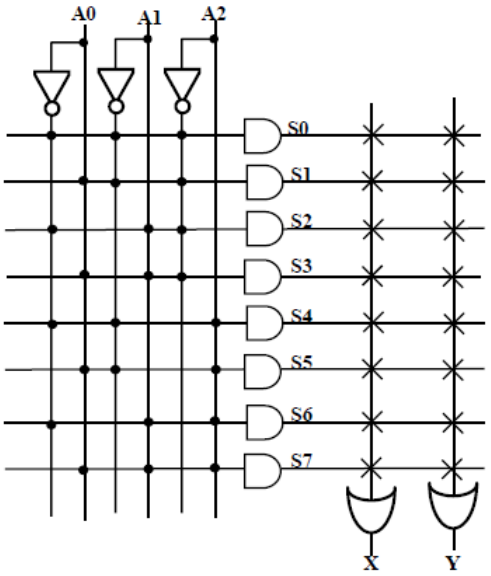


Figure 1.12 – PROM, Structure à ET fixe et OU Programmable

exemple : Soit les PLD des figure 1.13 et de la figure 1.14. Donnez les fonctions logiques des sorties X et Y pour la PROM et la PAL programmée.

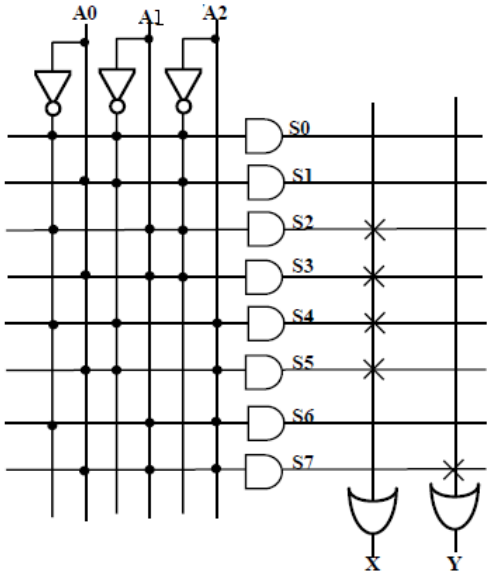


Figure 1.13 – Exercice PROM

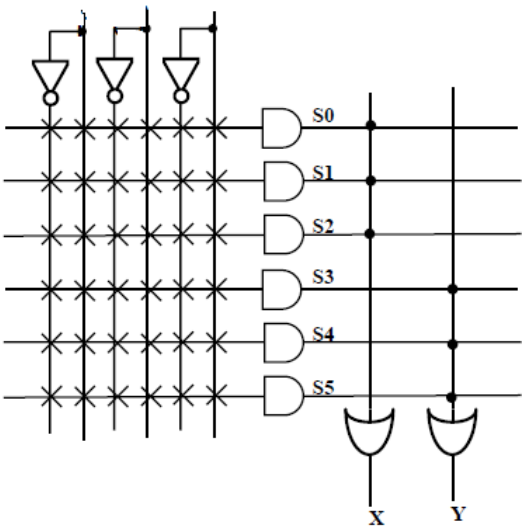


Figure 1.14 – Exercice PAL

1.4 Les circuits PAL

1.4.1 Présentation

Ce sont les premiers circuits programmables à être utilisés pour réaliser des fonctions logiques. Le constructeur AMD a développé ce type de circuit il y a près de 20 ans. Ils possèdent des matrices « ET » programmables et des matrices « OU » fixes. La fusion des fusibles est obtenue en appliquant

à leurs bornes une tension de 11,5 V pendant 10 à 50 ηS (leur tension de fonctionnement est environ de 5V). Cette opération est bien sûr effectuée en utilisant un programmateur adapté. Certaines broches de ces circuits peuvent être utilisées aussi bien en entrée qu'en sortie grâce à un système de logique 3 états. La commande de cette dernière est configurée au moment de la programmation. La structure de sortie permet aussi de réinjecter les sorties en entrée (Feed-back).

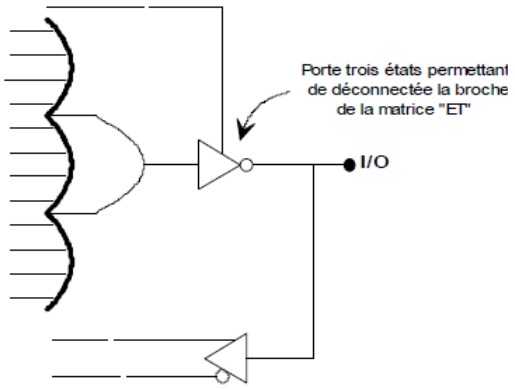


Figure 1.15 – configuration des entrée/sortie

Selon le type de PAL la structure de sortie peut être constituée d'une porte « NON », d'une porte « OU » Exclusive , d'une bascule « D » ou d'une combinaison des trois. Le nombre d'entrées et de sorties est lui aussi lié à la référence du PAL.

1.4.2 Référence des PAL (selon AMD)

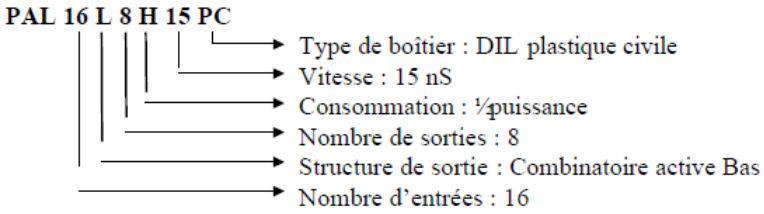
Les diverses possibilités de ces circuits et leur standardisation ont conduit les constructeurs à définir une nomenclature permettant de décoder assez facilement la référence des PALs.

Lettre(s) Code(s)	Structure de sortie
L	Combinatoire active bas
H	Combinatoire active haut
C	Combinatoire complémentaire
R	Registre synchrone (D)
RA	Registre asynchrone
X	Registre et OU exclusif
V	Versatile

Remarque :

- Le nombre d'entrées varie entre 10 et 22.
- Le nombre de sorties varie entre 1 et 10.
- La puissance est indiquée par une lettre code.
- La vitesse indique le temps de propagation en nS.
- Les versions versatiles ont une cellule de sortie programmable permettant d'obtenir n'importe quel autre type de structure de sortie (L, H, R ...).
- Les versions CMOS (CE) sont effaçables électriquement. Les fusibles sont remplacés par des transistors de type MOS FET. Ce ne sont ni plus ni moins que des « GALs ».

Exemple :



1.4.3 Exemple de PAL

Nous n'étudierons pas toutes les sortes de PAL car leur nombre est assez conséquent, mais nous intéresserons à ceux qui sont couramment utilisé.

1.4.3.1 Le PAL 16L8

Ce type de circuit est uniquement constitué de logique combinatoire. Il possède 20 broches (figure 1.17) agencées de la façon suivante :

- 10 broches configurables uniquement en entrée
- 2 broches configurables uniquement en sortie
- 6 broches configurables en entrée et en sortie
- 2 broches d'alimentation.

L'ensemble des sorties provient de portes 3 états inverseuses (figure 1.15). L'état haute impédance peut être commandée par l'ensemble des entrées. Chaque porte de la matrice « OU » possède 7 entrées. Ceci signifie que chaque sortie peut résulter, au maximum, d'une fonction « OU » entre 7 termes produits. Chaque porte de la matrice « ET » possède 32 entrées. Ceci signifie que chaque terme produit peut résulter, au maximum, d'une fonction « ET » entre 16 variables et leurs compléments.

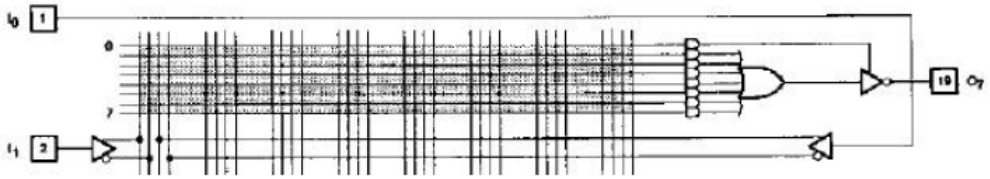


Figure 1.16 – Configuration partielle interne du PAL 16L8

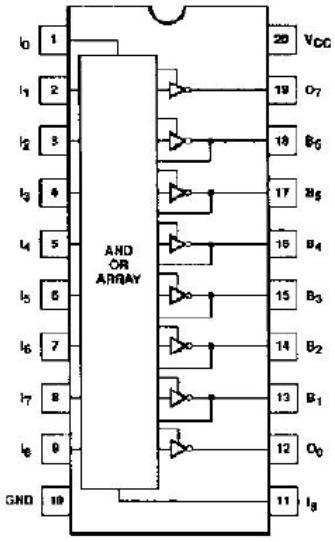


Figure 1.17 – Brochage du PAL 16L8

1.4.3.2 Le PAL 16R8

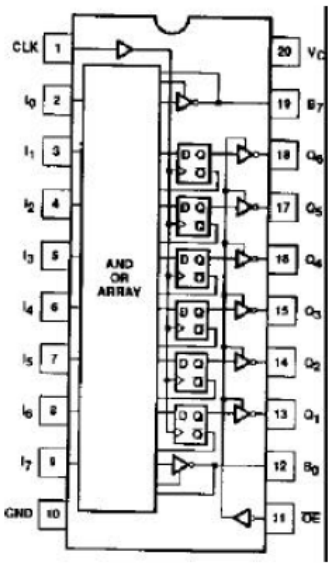


Figure 1.18 – Brochage du PAL 16R6

Ce type de circuit est constitué de logique combinatoire et séquentielle. Il possède 20 broches agencées de la façon suivante :

- 8 broches (n° 2 à 9) configurables uniquement en entrée
- 1 broche (n° 1) d'entrée d'horloge de l'ensemble des 8 bascules D
- 1 broche (n° 11) de validation des 8 sorties (output enable)
- 8 broches (n° 12 à 19) configurables en sortie et pouvant être réinjecter en entrée
- 2 broches d'alimentation (n° 10 et 20).

L'ensemble des sorties provient de portes 3 états inverseuses provenant elles-mêmes de bascules D (figure 1.18). L'état haute impédance est commandée par l'entrée OE (broche n°11). Chaque porte de la matrice « OU » possède 7 entrées. Ceci signifie que chaque sortie peut résulter, au maximum, d'une fonction « OU » entre 7 termes produits. Chaque porte de la matrice « ET » possède 32 entrées. Ceci signifie que chaque terme produit peut résulter, au maximum, d'une fonction « ET » entre 16 variables et leurs compléments. En résumé on peut dire que les huit sorties de ce circuit proviennent d'un registre trois états. La mise en haute impédance de ce dernier est commandée par la broche OE et la mémorisation est activée par les fronts montants de l'horloge CLK.

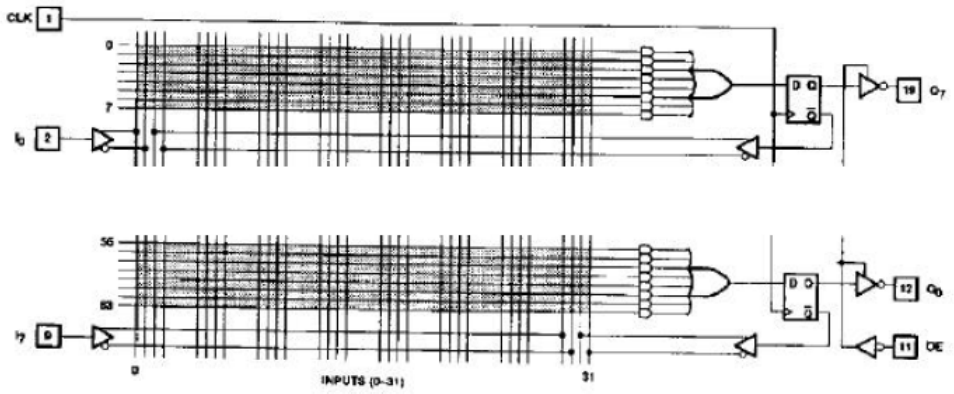


Figure 1.19 – Configuration partielle interne du PAL 16R8

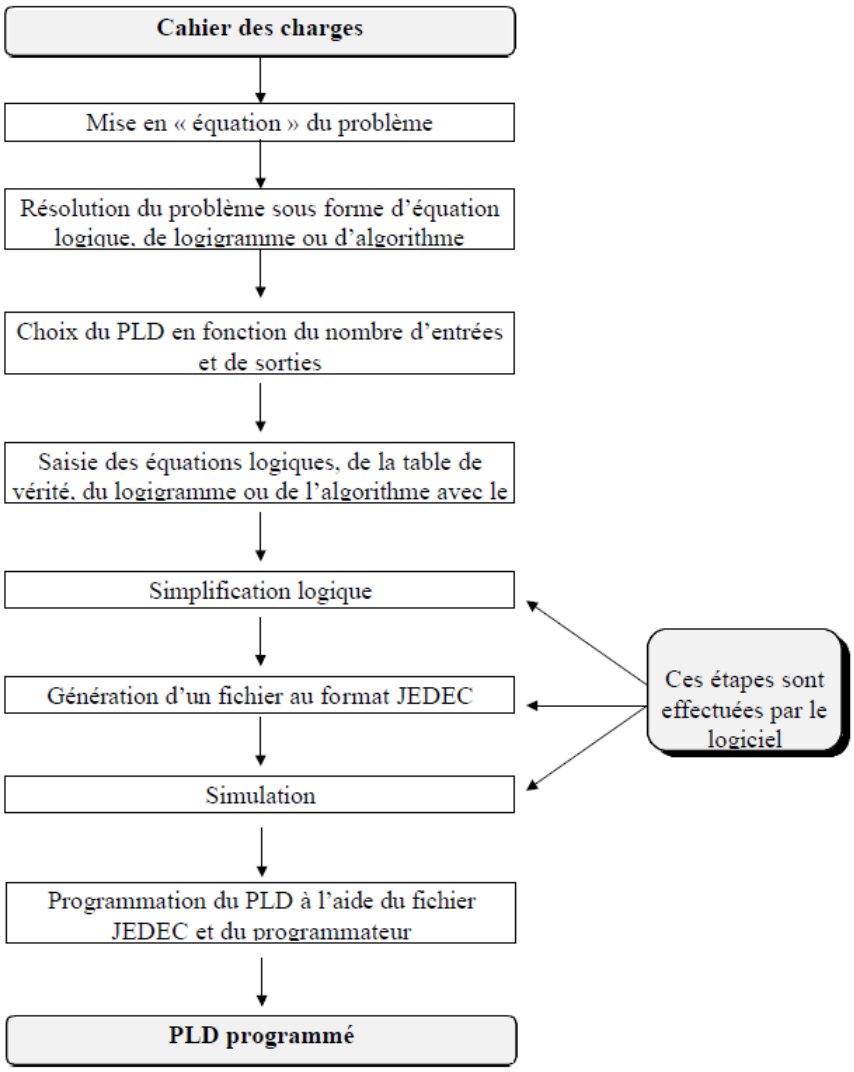
1.5 Programmation des PLD

La programmation des PLDs nécessite un logiciel adapté pour le développement du programme et un programmateur permettant de « griller » le circuit. En outre il est conseillé de suivre la démarche décrite par l'organigramme suivant :

* Le logiciel de développement permet de simplifier les équations et de générer un fichier JEDEC à partir des données rentrées par l'opérateur. Il simule aussi le fonctionnement du PLD avec le programme obtenu. Le logiciel « PALASM » développé par « AMD » permet d'effectuer ces opérations pour la programmation de PAL ou de GAL.

* Le fichier JEDEC est un ensemble de données binaires indiquant au programmateur les fusibles à « griller ».

* Le programmateur permet de « griller » les fusibles du PLD en fonctions des données du fichier JEDEC. Il est en général associé à un logiciel de pilotage. Les programmateurs utilisés sont les mêmes que ceux permettant la programmation des EPROM.


Cahier des charges
Mise en « équation » du problème
Résolution du problème sous forme d'équation logique, de logigramme ou d'algorithme
Choix du PLD en fonction du nombre d'entrées et de sorties
Saisie des équations logiques, de la table de vérité, du logigramme ou de l'algorithme avec le
Simplification logique
Génération d'un fichier au format JEDEC
Simulation
Programmation du PLD à l'aide du fichier JEDEC et du programmateur
PLD programmé
Ces étapes sont effectuées par le logiciel

CHAPITRE 2

Initiation au Language VHDL

Introduction

Pourquoi avoir créé un langage de description de structures électroniques (**HDL Hardware Description language**) **VHDL** ? L'abréviation **VHDL** signifie **VHSIC H**ardware **D**escription **L**anguage (**VHSIC** : **V**ery **H**igh **S**peed **I**ntegrated **C**ircuit). Ce langage a été écrit dans les années 70 pour réaliser la simulation de circuits électroniques. On l'a ensuite étendu en lui rajoutant des extensions pour permettre la conception (synthèse) de circuits logiques programmables (**PLD P**rogrammable **L**ogic **D**evice). Auparavant pour décrire le fonctionnement d'un circuit électronique programmable les techniciens et les ingénieurs utilisaient des langages de bas niveau (**ABEL, PALASM, ORCAD/PLD**,..) ou plus simplement un outil de saisie de schémas. Actuellement la densité de fonctions logiques (portes et bascules) intégrée dans les PLDs est telle (plusieurs milliers de portes voire millions de portes) qu'il n'est plus possible d'utiliser les outils d'hier pour développer les circuits d'aujourd'hui. Les sociétés de développement et les ingénieurs ont voulu s'affranchir des contraintes technologiques des circuits. Ils ont donc créé des langages dits de haut niveau à savoir **VHD**L et **VERILOG**. Ces deux langages font abstraction des contraintes technologies des circuits **PLDs**. Ils permettent au code écrit d'être portable, c'est à dire qu'une description écrite pour un circuit peut être facilement utilisée pour un autre circuit. Il faut avoir à l'esprit que ces langages dits de haut niveau permettent de matérialiser les structures électroniques d'un circuit. En effet les instructions écrites dans ces langages se traduisent par une configuration logique de portes et de bascules

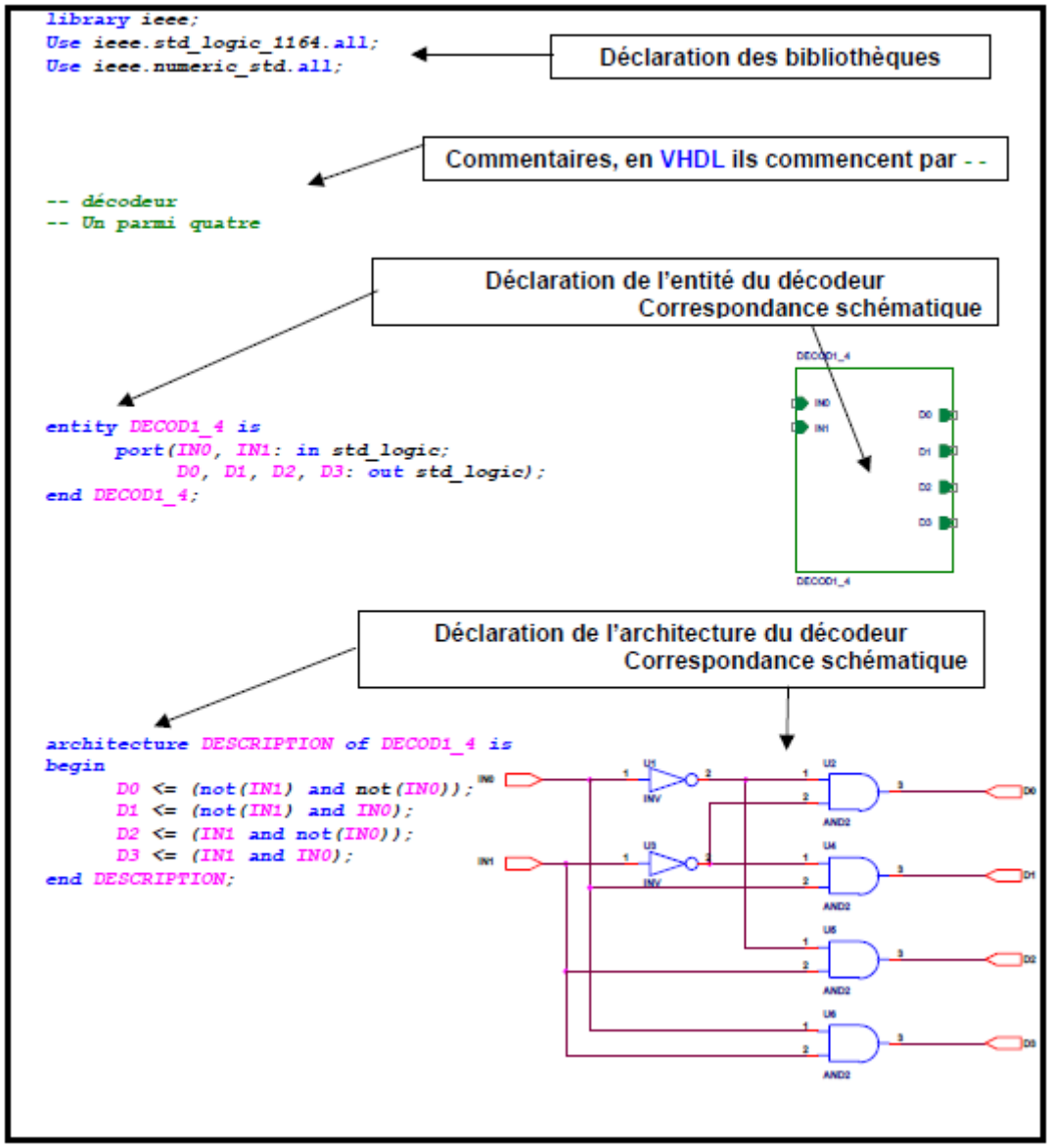
qui est intégrée à l'intérieur des circuits PLDs. C'est pour cela que je préfère parler de description **VHDL** ou **VERILOG** que de langage. Dans ce polycopié, je m'intéresserai seulement à **VHDL** et aux fonctionnalités de base de celui-ci lors des phases de conception ou synthèse (c'est à dire à la conception de **PLDs**).

2.1 Structure d'une description VHDL simple

Une description VHDL est composée de 2 parties *indissociables* à savoir :

- L'*entité* (**ENTITY**), elle définit les entrées et sorties.
- L'*architecture* (**ARCHITECTURE**), elle contient les instructions VHDL permettant de réaliser le fonctionnement attendu.

Exemple : Un décodeur 1 parmi 4



2.1.1 Déclaration des bibliothèques

Toute description VHDL utilisée pour la synthèse a besoin de bibliothèques. L'**IEEE** (**I**nstitut of **E**lectrical and **E**lectronics **E**ngineers) les a normalisées et plus particulièrement la bibliothèque **IEEE1164**. Elles contiennent les définitions des types de signaux électroniques, des fonctions et sous programmes permettant de réaliser des opérations arithmétiques et logiques,...

```
Library ieee;
Use ieee.std_logic_1164.all;
Use ieee.numeric_std.all;
Use ieee.std_logic_unsigned.all;
-- cette dernière bibliothèque est souvent utilisée pour l'écriture de compteurs
```

La directive Use permet de sélectionner les bibliothèques à utiliser.

2.1.2 Déclaration de l'entité et des entrées/sorties (I/O)

Elle permet de définir le NOM de la description VHDL ainsi que les entrées et sorties utilisées, l'instruction qui les définit c'est port :

Syntaxe :

```
entity NOM_DE_L_ENTITE is
      port ( Description des signaux d'entrées /sorties …);
end NOM_DE_L_ENTITE;
```

Exemple :

```
entity SEQUENCEMENT is
port (
      CLOCK      : in std_logic;
      RESET      : in std_logic;
      Q          : out std_logic_vector(1 downto 0)
      );
end SEQUENCEMENT;
```

Remarque : Après la dernière définition de signal de l'instruction port il ne faut jamais mettre de point virgule.

L'instruction **port**

```
Syntaxe:   NOM_DU_SIGNAL   : sens     type;

Exemple:   CLOCK: in       std_logic;
           BUS   : out     std_logic_vector (7 downto 0);
```

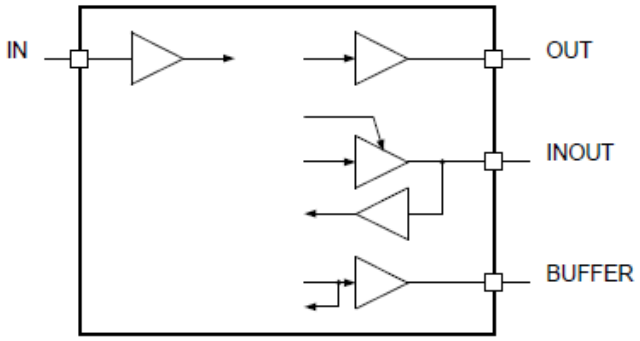
On doit définir pour chaque signal : le **NOM DU SIGNAL**, le **sens** et le **type**.

2.1.2.1 NOM DU SIGNAL

Il est composé de caractères, le premier caractère doit être une lettre, sa longueur est quelconque, mais elle ne doit pas dépasser une ligne de code. VHDL n'est pas sensible à la « casse », c'est à dire qu'il ne fait pas la distinction entre les majuscules et les minuscules.

2.1.2.2 Le sens du signal

- **in** : pour un signal en entrée (monodirectionnelle)
- **out** : pour un signal en sortie (monodirectionnelle)
- **inout** : pour un signal en entrée sortie (bidirectionnelle)
- **buffer** : pour un signal en sortie mais utilisé comme entrée dans la description(Sortie rebouclée)



2.1.2.3 Le type

Le TYPE utilisé pour les signaux d'entrées / sorties est :

- le **std logic** pour un signal
- le **std logic vector** pour un bus composé de plusieurs signaux

Par exemple un bus bidirectionnel de 5 bits s'écrira : Où **LATCH(4)**

```
LATCH : inout std_logic_vector (4 downto 0) ;
```

correspond au **MSB** et **LATCH(0)** correspond au **LSB**.

Les valeurs que peuvent prendre un signal de type **std logic** sont :

- std_ulogic :

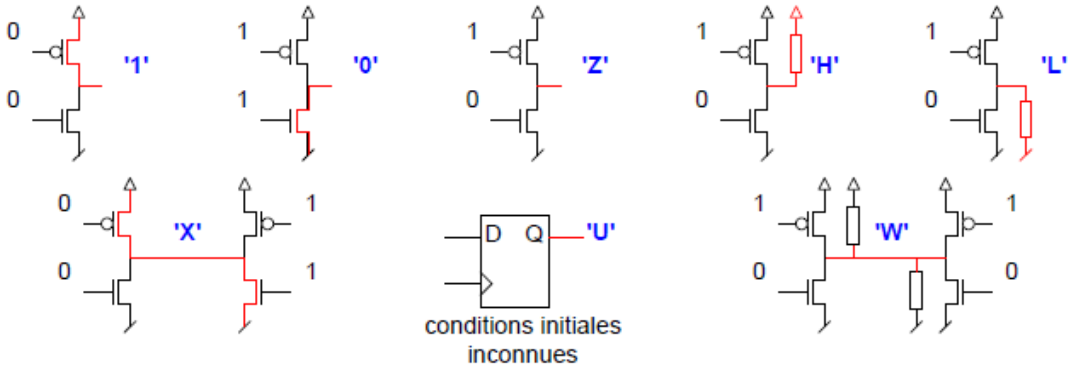
```
TYPE std_ulogic IS (    'U',    -- Uninitialized
                        'X',    -- Forcing  Unknown
                        '0',    -- Forcing  0
                        '1',    -- Forcing  1
                        'Z',    -- High Impedance
                        'W',    -- Weak     Unknown
                        'L',    -- Weak     0
                        'H',    -- Weak     1
                        '-'     -- Don't care);
```

9 états définis par la librairie standard

std_logic_1164

- std_logic :

```
SUBTYPE std_logic IS resolved std_ulogic;
```



2.1.2.4 Exemples de description d'entités

```
entity COUNT is
    port(CLK, RST:  in std_logic; CNT:  inout std_logic_vector(2 downto 0));
end COUNT;


entity MAGNITUDE is
     port( A, B: in std_logic_vector(7 downto 0);
     AGRB: buffer std_logic));
end MAGNITUDE;


entity 7SEG is
     port (
           A     : in std_logic;
           B     : in std_logic;
           C     : in std_logic;
           D     : in std_logic;
           SA    : out std_logic;
           SB    : out std_logic;
           SC    : out std_logic;
           SD    : out std_logic;
           SE    : out std_logic;
           SF    : out std_logic;
           SG    : out std_logic );
end 7SEG;


entity IMAGE3 is
     port (CLK : in std_logic; S,I : out std_logic);
end IMAGE3;


entity COMP4BIT is
     port (A,B :in std_logic_vector(3 downto 0);
     PLUS,MOINS,EGAL :out std_logic);
end COMP4BIT;


entity COMPT_4 is
     port (H,R :in std_logic;
     Q :out std_logic_vector(3 downto 0));
end COMPT_4;


entity DECAL_D is
     port (H,R,IN_SERIE :in std_logic;
     OUT_SERIE :out std_logic);
end DECAL_D;
```

2.1.3 Déclaration de l'architecture correspondante à l'entité

L'architecture décrit le fonctionnement souhaité pour un circuit ou une partie du circuit. En effet le fonctionnement d'un circuit est généralement décrit par plusieurs modules VHDL. Il faut comprendre par module le couple ENTITE/ARCHITECTURE. Dans le cas de simples PLDs on trouve souvent un seul module. L'architecture établit à travers les instructions les relations entre les entrées et les sorties. On peut avoir un fonctionnement purement combinatoire, séquentiel voire les deux séquentiel et combinatoire.

Exemple :

```
entity PORTES is
      port (A,B :in std_logic;
      Y1,Y2,Y3,Y4,Y5,Y6,Y7:out std_logic);
end PORTES;

architecture DESCRIPTION of PORTES is
begin
      Y1 <= A and B;
      Y2 <= A or B;
      Y3 <= A xor B;
      Y4 <= not A;
      Y5 <= A nand B;
      Y6 <= A nor B;
      Y7 <= not(A xor B);
end DESCRIPTION;
```

2.2 Les instructions de base (mode « concurrent »), logique combinatoire

Qu'est ce que le mode « concurrent » ? Pour une description VHDL toutes les instructions sont évaluées et affectent les signaux de sortie en même temps. L'ordre dans lequel elles sont écrites n'a aucune importance. En effet la description génère des structures électroniques, c'est la grande différence entre une description VHDL et un langage informatique classique. Dans un système à microprocesseur, les instructions sont exécutées les unes à la suite des autres.

Avec VHDL il faut essayer de penser à la structure qui va être générée par le synthétiseur pour écrire une bonne description, cela n'est pas toujours évident.

Exemple : Pour le décodeur 1 parmi 4, l'ordre dans lequel seront écrites les instructions n'a aucune importance.

```
architecture DESCRIPTION of DECOD1_4 is
begin
D0 <= (not(IN1) and not(IN0));      -- première instruction
D1 <= (not(IN1) and IN0);           -- deuxième instruction
D2 <= (IN1 and not(IN0));           -- troisième instruction
D3 <= (IN1 and IN0);                -- quatrième instruction
end DESCRIPTION;
```

L'architecture ci dessous est équivalente :

```
architecture DESCRIPTION of DECOD1_4 is
begin
D1 <= (not(IN1) and IN0);           -- deuxième instruction
D2 <= (IN1 and not(IN0));           -- troisième instruction
D0 <= (not(IN1) AND not(IN0));      -- première instruction
D3 <= (IN1 AND IN0);                -- quatrième instruction
end DESCRIPTION;
```

L'instruction définissant l'état de D0 à été déplacée à la troisième ligne, la synthèse de cette architecture est équivalente à la première.

2.2.1 Les opérateurs

2.2.1.1 L'affectation simple : <=

Dans une description VHDL, c'est certainement l'opérateur le plus utilisé. En effet il permet de modifier l'état d'un signal en fonction d'autres signaux et/ou d'autres opérateurs.

Exemple avec des portes logiques : Exemple avec des portes logiques :

```
S1 <= E2 and E1 ;
```

Pour les signaux composés de plusieurs bits on utilise les guillemets " ... " , les bases numériques utilisées pour les bus peuvent être :

```
BINAIRE,    exemple : BUS <= "1001" ;      -- BUS = 9 en décimal
HEXA,       exemple : BUS <= X"9" ;  -- BUS = 9 en décimal
OCTAL,      exemple : BUS <= O"11" ;       -- BUS = 9 en décimal
```

Remarque :

La base décimale ne peut pas être utilisée lors de l'affectation de signaux. On peut seulement l'utiliser avec certains opérateurs, comme + et - pour réaliser des compteurs (voir le chapitre sur les compteurs).

Exemple :

```
Library ieee;
Use ieee.std_logic_1164.all;

entity AFFEC is
port (
      E1,E2              : in std_logic;
      BUS1,BUS2,BUS3     : out std_logic_vector(3 downto 0);
      S1,S2,S3,S4        : out std_logic);
end AFFEC;

architecture DESCRIPTION of AFFEC is

begin
S1 <= '1'; -- S1 = 1
S2 <= '0'; -- S2 = 0
S3 <= E1;  -- S3 = E1
S4 <= '1' when (E2 ='1') else 'Z'; -- S4 = 1 si E2=1 sinon S4
                                   -- prend la valeur haute impédance
BUS1 <= "1000";                    -- BUS1 = "1000"
BUS2 <= E1 & E2 & "10";            -- BUS2 = E1 & E2 & 10
BUS3 <= x"A";                      -- valeur en HEXA -> BUS3 = 10(déc)

end DESCRIPTION;
```

2.2.1.2 Opérateur de concaténation

Cet opérateur permet de joindre des signaux entre eux.

Exemple :

```
--   Soit A et B de type 3 bits et S1 de type 8 bits
--   A = "001"  et B ="110"
S1 <= A & B & "01" ;
-- S1 prendra la valeur suivante après cette affectation
-- S1 = "00111001"
```

2.2.1.3 Opérateurs logiques

Opérateur	VHDL
ET	and
NON ET	nand
OU	or
NON OU	nor
OU EXCLUSIF	xor
NON OU EXCLUSIF	xnor
NON	not
DECALAGE A GAUCHE	sll
DECALAGE A DROITE	srl
ROTATION A GAUCHE	rol
ROTATION A DROITE	ror

Remarque :

Pour réaliser des décalages logiques en synthèse logique, il est préférable d'utiliser les instructions suivantes

Décalage à droite :

```
-- Si A est de type std_logic_vector(7 downto 0)
S1 <= '0' & A(7 downto 1); -- décalage d'un bit à droite
S1 <= "000" & A(7 downto 3); -- décalage de trois bits à droite
```

Décalage à gauche :

```
-- Si A est de type std_logic_vector(7 downto 0)
S1 <= A(6 downto 0) & '0'; -- décalage d'un bit à gauche
S1 <= A(4 downto 0) & "000"; -- décalage de trois bits à gauche
```

2.2.1.4 Opérateurs arithmétiques

Opérateur	VHDL
ADDITION	+
SOUSTRACTION	-
MULTIPLICATION	*
DIVISION	/

Remarque

Pour pouvoir utiliser les opérateurs ci-dessus il faut rajouter les bibliothèques suivantes au début du fichier VHDL :

```
Use ieee.numeric_std.all ;
Use ieee.std_logic_arith.all ;
```

2.2.1.5 Opérateurs relationnels

Ils permettent de modifier l'état d'un signal ou de signaux suivant le résultat d'un test ou d'une condition. En logique combinatoire ils sont souvent utilisés avec les instructions :

- when ... else ...
- with ... Select ...

Opérateur	VHDL
Egal	=
Non égal	/=
Inférieur	<
Inférieur ou égal	<=
Supérieur	>
Supérieur ou égal	>=

2.2.2 Les instructions du mode « concurrent »

2.2.2.1 Affectation conditionnelle

Cette instruction modifie l'état d'un signal suivant le résultat d'une condition logique entre un ou des signaux, valeurs, constantes.

```
SIGNAL <= expression when condition
          [else expression when condition]
          [else expression];
```

Remarque : l'instruction [else expression] n'est pas obligatoire mais elle fortement conseillée, elle permet de définir la valeur du SIGNAL dans le cas où la condition n'est pas remplie.

Exemple :

```
-- Structure évoluée d'un multiplexeur 4 vers 1
S2 <= E1 when (SEL="00" ) else
      E2 when (SEL="01" ) else
      E3 when (SEL="10" ) else
      E4 when (SEL="11" )
      else '0';
```

2.2.2.2 Affectation sélective

Cette instruction permet d'affecter différentes valeurs à un signal, selon les valeurs prises par un signal dit de sélection.

```
with SIGNAL_DE_SELECTION select
     SIGNAL <=      expression when valeur_de_selection,
                    [expression when valeur_de_selection,]
                    [expression when others];
```

Remarque : l'instruction **expression when others** n'est pas obligatoire mais fortement conseillée, elle permet de définir la valeur du SIGNAL dans le cas où la condition n'est pas remplie.

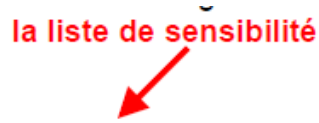
```
-- Multiplexeur 4 vers 1
with SEL select
    S2 <=   E1  when "00",
            E2  when "01",
            E3  when "10",
            E4  when "11",
            '0' when others;
```

Remarque : when others est nécessaire car il faut toujours définir les autres cas du signal de sélection pour prendre en compte toutes les valeurs possibles de celui-ci.

2.3 Les instructions du mode séquentiel

2.3.1 Définition d'un PROCESS

Un **process** est une partie de la description d'un circuit dans laquelle les instructions sont exécutées séquentiellement c'est à dire les unes à la suite des autres. Il permet d'effectuer des opérations sur les signaux en utilisant les instructions standard de la programmation structurée comme dans les systèmes à microprocesseurs. L'exécution d'un **process** est déclenchée par un ou des changements d'états de signaux logiques. Le nom de ces signaux est défini dans la **liste de sensibilité** lors de la déclaration du **process**.




```
[Nom_du_process :] process(Liste_de_sensibilité_nom_des_signaux)
Begin
     -- instructions du process
end process [Nom_du_process] ;
```

Remarque : Le nom du process entre crochet est facultatif, mais il peut être très utile pour repérer un process parmi d'autres lors de phases de mise au point ou de simulations.

Règles de fonctionnement d'un process :

- L'exécution d'un process a lieu à chaque changement d'état d'un signal de la liste de sensibilité.
- Les instructions du process s'exécutent séquentiellement.
- Les changements d'état des signaux par les instructions du process sont pris en compte à la fin du process.

2.3.2 Les deux principales structures utilisées dans un process

L'assignation conditionnelle

```
if condition then
     instructions
[elsif condition then instructions]
[else instructions]
end if ;


Exemple:
if (RESET='1') then SORTIE <= "0000";
end if ;
```

L'assignation sélective

```
case signal_de_slection is
when valeur_de_sélection => instructions
[when others => instructions]
end case;


Exemple:
case SEL is
          when "000" => S1 <= E1;
          when "001" => S1 <= '0';
          when "010" | "011" => S1 <='1';
          -- La barre | permet de réaliser
          -- un ou logique entre les deux
          -- valeurs "010" et "011"
          when others => S1 <= '0';
end case;
```

Remarque : ne pas confondre => (implique) et <= (affecte).

2.3.3 Exemples de process

Déclaration d'une bascule D

```
Library ieee;
Use ieee.std_logic_1164.all;
Use ieee.numeric_std.all;
Use ieee.std_logic_unsigned.all;
entity BASCULED is
      port (
            D,CLK : in std_logic;
            S     : out std_logic);
end BASCULED;
architecture DESCRIPTION of BASCULED is
begin
      PRO_BASCULED : process (CLK)
      begin
            if (CLK'event and CLK ='1') then
                  S <= D;
            end if;
      end process PRO_BASCULED;
end DESCRIPTION;
```

Commentaires

- Seul le signal CLK fait partie de la liste de sensibilité. D'après les règles de fonctionnement énoncées précédemment, seul un changement d'état du signal CLK va déclencher le process et par conséquent évaluer les instructions de celui-ci.
- L'instruction if $(CLK'eventandCLK =' 1')$ then permet de détecter un front montant du signal CLK. La détection de front est réalisée par l'attribut event appliqué à l'horloge CLK. Si on veut un déclenchement sur un front descendant, il faut écrire l'instruction suivante : if $(CLK'eventandCLK =' 0')$.
- Les bibliothèques IEEE possèdent deux instructions permettant de détecter les fronts montants) rising edge(CLK) ou descendants falling edge(CLK).
- Si la condition est remplie alors le signal de sortie S sera affecté avec la valeur du signal d'entrée D.

2.4 Les compteurs

Ils sont très utilisés dans les descriptions VHDL. L'écriture d'un compteur peut être très simple comme très compliquée. Ils font appels aux process.

Compteur simple

```
Library ieee;
Use ieee.std_logic_1164.all;
Use ieee.numeric_std.all;
Use ieee.std_logic_unsigned.all;

entity CMP4BITS is
PORT (
      CLOCK : in std_logic;
      Q     : inout std_logic_vector(3 downto 0));
end CMP4BITS;

architecture DESCRIPTION of CMP4BITS is
begin
  process (CLOCK)
  begin
    if (CLOCK ='1' and CLOCK'event) then
      Q <= Q + 1;
    end if;
  end process;
end DESCRIPTION;
```

2.5 Installation du logiciel de Programmation VHDL

Le Logiciel de programmation utilisé est **Quartus II** version 10.1. (voir documents annexes pour les démarches d'installation, et d'utilisation)

2.6 Installation du logiciel de Simulation

Le Logiciel de simulation utilisé est **ModelSim-Altera** version 6.6c, un outils complémentaire de **Quartus II**. (voir documents annexes pour les démarches d'utilisation)

2.7 Travaux pratiques de Programmation et de Simulation

- Synthèse de différentes fonction logique de Base
- Synthèse d'un décodeur 1 parmi quatre
- Synthèse d'un décodeur BCD/7 Segments